

Exerciții — o platformă de la zero, cu atelierul înainte

16 exerciții · două servicii, un client, infrastructura lor — toate prin agent · lecția 3

Terenul de lucru: un folder **nou și gol**, lângă `ms\` — de exemplu `C:\Projects\teste claudel\velo`. Zero legătură cu platforma ta: alte repo-uri, alt cluster, alt domeniu. Îți trebuie **Rancher Desktop cu Kubernetes pornit** (contextul `rancher-desktop` e deja în kubeconfig-ul tău), Docker Hub și `gh`. **Ce se schimbă față de fișele 1 și 2:** acolo platforma exista și **tu știai răspunsul înainte să întrebi**. Aici nu există nimic și nu știe nimeni răspunsul — nici tu, nici agentul. De aceea fiecare verificare din fișa asta se scrie **înainte** de lucrul pe care îl verifică: gardul înainte de primul fișier, harta înainte de teritoriu, testul înainte de cod, lanțul înainte de al doilea serviciu.

Platforma: `velo` — stații de biciclete, cu capacitate și număr de biciclete; o cursă pornește dintr-o stație și se închide într-una. Patru repo-uri publice, create de tine la exercițiul 5:

Repo	Ce e
<code>statii-api</code>	stațiile: capacitate, câte biciclete sunt acum, scoate una / pune una
<code>curse-api</code>	cursele: pornire (scoate o bicicletă din stația A), închidere (o pune în stația B)
<code>velo-web</code>	clientul: stațiile, cursele deschise, pornește / închide o cursă
<code>velo-gitops</code>	singurul repo citit de ArgoCD: infra, chart-uri, Application-uri

Domeniul e ales pentru un singur motiv: are o regulă verificabilă cu două `curl`-uri — **suma bicicletelor din stații + cursele deschise = 30, întotdeauna**. Fără ea, nicio poartă din fișă n-ar avea ce să prindă.

Partea I — Atelierul înainte de orice: nu există nicio linie scrisă

Gardul pe un folder gol

1

Creează folderul, intră în el, pornește `claude`. Înainte de orice altceva, `/permissions : deny` pe `kubectll apply / delete / scale / patch`, `helm install / upgrade / uninstall`, `argocd app sync / delete`, `git push`, `docker push`, `gh repo delete`. Apoi cere-i:

Instaleaza ArgoCD in clusterul rancher-desktop.

Și imediat după:

Ce contexte are kubectll, care e activ si ce ruleaza in el?

Gata când: prima a fost refuzată cu motivul pe ecran, a doua a răspuns — iar contextul activ e `rancher-desktop`, **nu** `nucserver`. Dacă e `nucserver`, oprește-te aici: ai două clustere în același kubeconfig și agentul nu știe care e cel adevărat. Schimbă contextul cu mâna, apoi continuă.

De ce începe fișa cu un gard pe nimic: gardul nu e despre ce există în folder, e despre cine are voie ce. Pe `ms\` l-ai pus după un an de platformă. Aici îl pui pe un folder gol, și e primul fișier din el.

Harta înainte de teritoriu

Scrie `CLAUDE.md` cu mâna ta, înainte să existe vreun repo. Nu e un rezumat — e un **contract**: cele patru repo-uri și rolul fiecăruia · namespace-ul `velo` · **un singur repo GitOps** · convenția de tag (sha7, scris de ci-bot, cu ghilimele) · contextul `kubect1` permis, și regula că orice alt context înseamnă *oprește-te* · regula cu cele 30 de biciclete. Apoi sesiune nouă și:

Ce servicii are platforma, în ce namespace rulează, și cine scrie tag-ul din `values.yaml`? Și câte repo-uri git sunt acum în folderul asta?

Gata când: la primele trei a răspuns corect din `CLAUDE.md`, cu discul gol — iar la a patra a spus **zero**, nu a inventat. Harta spune patru, podeaua spune zero, și agentul trebuie să le spună pe amândouă.

Ce e diferit față de CLAUDE.md-ul din ms\: acela descria ceva construit; ăsta decide ceva neconstruit. **Tot ce nu scrie aici, agentul decide singur** — vezi la exercițiul 3 cât de mult.

Planul, în mod plan

Shift+Tab până la **plan**. Apoi:

Propune structura platformei `velo` conform `CLAUDE.md`: ce conține fiecare repo, stack-ul celor două servicii (acelasi limbaj pentru amandoua), baza de date, și drumul unei imagini din CI până în pod. Nu scrie niciun fisier.

Citește planul până la capăt. Caută în el **un lucru pe care nu l-ai cerut și nu e în CLAUDE.md** — un Kafka, un Redis, un al doilea repo GitOps, un tag `latest`, un `namespace` în plus. E aproape sigur acolo.

Gata când: git status și dir arată același folder ca înainte, **și** ai găsit lucrul inventat. Pentru el ai două opțiuni, și decizi tu, nu agentul: intră în `CLAUDE.md` cu `#`, sau îi spui să-l scoată din plan. Dacă planul e greșit de la rădăcină, Esc Esc și reformulează cererea — nu cârpi planul.

De ce în mod plan: tot ce nu e în hartă e la latitudinea agentului. Planul ți-l arată pe o pagină, înainte să-l vezi răspândit în patruzeci de fișiere.

Regula domeniului, scrisă înainte de API

Adaugă regula în `CLAUDE.md` fără să deschizi fișierul:

```
# Invariant: suma bicicletelor din statii + cursele deschise = 30,
intotdeauna. Orice ruta care schimba numarul de biciclete il pastreaza.
```

Apoi, tot în mod plan:

Scrie `CONTRACT.md`: rutele HTTP ale celor două servicii, ce primesc, ce întorc. Pentru fiecare ruta care schimba numărul de biciclete, scrie ce se întâmplă când ar încălca invariantul: stație goală, stație plină, cursa închisă de două ori, stație inexistentă.

Gata când: fiecare rută care scrie are **cazul ei de refuz**, explicit. Dacă una n-are — de obicei închiderea de două ori a aceleiași curse — tocmai ai găsit primul bug al platformei, înainte de prima linie de cod. Completează contractul, nu codul.

Aici se întoarce ordinea obișnuită. De obicei regula se descoperă din bug, după ce linia e scrisă. Aici linia nu există încă, și regula e deja în două fișiere.

Granița mâinii tale. Regula cursului nu se schimbă: *agentul nu atinge clusterul; agentul schimbă repo-ul; clusterul se schimbă singur*. Tot ce faci tu cu mâna în fișa asta este lista de mai jos, și **numai** ea: gh repo create de patru ori · secretele de CI în GitHub, la Settings → Secrets · trei linii în hosts · helm install pentru ArgoCD · kubectl apply pe bootstrap/root.yaml. Două apply-uri în toată fișa. Exercițiul 16 le numără.

Partea II — Scheletul: un singur serviciu, cap-coadă, înainte de al doilea

Testul înainte de cod

5

Creează cele patru repo-uri pe GitHub, publice, și clonează-le în `velo\`. Apoi, în `statii-api`:

```
Scrive DOAR testele pentru statii-api, dupa CONTRACT.md: scoaterea
dintr-o statie cu 0 biciclete e refuzata, punerea intr-o statie plina e
refuzata, si totalul nu se schimba dupa orice succesiune de operatii.
Nicio implementare. Ruleaza-le: trebuie sa cada.
```

Gata când: testele cad — și cad **din motivul corect**. Citește mesajul fiecăruia: „implementarea lipsește / întoarce nimic” e corect; o eroare de sintaxă în test sau o dependență lipsă nu e un test care cade, e un test care nu rulează.

Implementarea, și poarta pe test

6

```
Implementeaza statii-api pana trec toate testele. Nu modifica
testele. Datele initiale: 3 statii, 30 de biciclete in total.
```

Rulează testele din sesiune, cu `!`, ca ieșirea să intre în conversație. Când sunt verzi, **poarta**: deschide tu codul, scoate cu mâna verificarea „statie goală”, rulează din nou.

Gata când: verde pe cod bun, roșu pe cod stricat, apoi `git diff` gol. Dacă pe cod stricat a rămas verde, nu codul era problema — testul era un afiș. Pune-l să-l repare pe test, nu pe cod. La final: `/init`, și compară ce a scris cu harta ta. Ce a scris și **nu poți verifica din fișiere** se taie.

Dockerfile și CI, cu două scanări înăuntru de la prima rulare

7

```
Scrive Dockerfile multi-stage si .github/workflows/ci.yml pentru
statii-api: teste, gitleaks pe repo, build image, trivy pe image
(CRITICAL opreste pipeline-ul), push pe Docker Hub cu tag sha7, apoi
bump-ul tag-ului in velo-gitops, la business/statii-api/values.yaml,
facut de ci-bot, cu tag-ul intre ghilimele. Secretele de CI nu le
scrii tu nicaieri: le pun eu in GitHub.
```

Pune cu mâna, în Settings → Secrets, tokenul de Docker Hub și un token cu drept de scriere **doar** pe `velo-gitops`. Scrie în `CLAUDE.md` data la care expiră. Push. Apoi **două porți**, pe un branch separat:

- un commit cu o linie `API_KEY=abc123def456` într-un fișier oarecare;
- un commit care schimbă base image-ul cu unul de acum doi ani.

Gata când: run-ul pe `main` e verde și imaginea e pe Docker Hub; run-ul cu secretul e **roșu pe gitleaks**; run-ul cu imaginea veche e **roșu pe trivy**. Trei run-uri, trei culori pe care le poți arăta. Apoi branch-ul se șterge.

Ghilimelele sunt în scriptul de bump, nu în values.yaml. Primul tag format numai din cifre va ajunge acolo peste câteva zile; cine le-a pus pe valoare și nu pe producător o află atunci. Scanările costă zece linii acum, pentru că pipeline-ul nu există încă. Pe un pipeline de un an costă o săptămână.

8

velo-gitops: ArgoCD instalat de tine, o singură dată — restul vine din repo

Instalează ArgoCD cu mâna, cu `helm install`, în `rancher-desktop`. E primul dintre cele două `apply`-uri ale fișei, chiar dacă se cheamă altfel. Apoi:

```
Scrie velo-gitops: bootstrap/root.yaml (App-of-Apps), argo-apps/ cu un
Application pentru infra si unul pentru statii-api, infra/ cu operatorul
cloudnative-pg si un Cluster Postgres cu o instanta in namespace-ul velo,
chart-ul statii-api cu values (image, tag intre ghilimele, datele de
conectare citite din secretul generat de operator, ingress pe
statii.velo.localhost). Nu rula nimic in cluster.
```

Al doilea `apply`: `kubectl apply -f bootstrap/root.yaml`. Trei linii în `hosts` pentru `statii /`
`curse / app.velo.localhost`. Apoi aștepti. Sesiunea asta e cea mai lungă din fișă: când `/context` trece
de jumătate, `/compact` înainte să continui.

Gata când: `kubectl get applications -n argocd` arată root și copiii **Synced / Healthy**, pod-ul
`statii-api` e Running, iar `curl http://statii.velo.localhost/statii` întoarce cele 3 stații cu
30 de biciclete. Nicio comandă în afara celor două nu a atins clusterul — dacă a fost nevoie de a treia,
notează-o: e o gaură în repo și o reparați acum, nu la exercițiul 16.

De ce operatorul generează secretul: `cloudnative-pg` creează singur credențialul bazei, iar serviciul îl citește
prin `secretKeyRef`. Deci în patru repo-uri publice **nu există niciun secret de comis** — nu pentru că ai
grijă, ci pentru că n-are de unde să apară. Asta e diferența dintre o regulă și o linie.

9

Lanțul, cap-coadă, pe un singur serviciu

Schimbă un lucru vizibil în `statii-api` — numele unei stații — commit, push. Apoi **nu faci nimic**, doar te uiți,
cu ceasul lângă tine, la fiecare verigă:

1. run-ul din GitHub Actions — ora la care s-a terminat;
2. tag-ul nou pe Docker Hub;
3. commit-ul lui ci-bot în `velo-gitops`, cu ghilimele în jurul tag-ului;
4. Application-ul: OutOfSync → Synced;
5. pod-ul nou, cu tag-ul nou în `kubectl describe`;
6. `curl` întoarce numele schimbat.

Gata când: ai cele șase ore notate, una sub alta, și `git log business/statii-api/values.yaml` în
`velo-gitops` are **un singur autor** pentru bump: ci-bot. Niciun tag scris de mâna ta, niciodată — altfel
lanțul are deja o verigă pe care o ții tu.

De ce aici, și nu la final: lista asta cu șase puncte e greu de făcut pe trei servicii și douăzeci și șapte de
Application-uri. Pe un serviciu, înainte să existe al doilea, durează douăzeci de minute. Toată fișa stă pe
ordinea asta.

Comanda /serviciu-nou

10

Înainte să scrii o linie din `curse-api`:

```
Fa o comanda /serviciu-nou <nume> care produce pentru un serviciu nou tot ce are statii-api in afara codului de domeniu: Dockerfile, pipeline-ul cu ambele scanari si bump-ul, chart-ul si Application-ul in velo-gitops, ingress-ul pe <nume>.velo.localhost, randul din CLAUDE.md. Pleaca de la ce exista in statii-api, nu inventa.
```

Rulează `/serviciu-nou curse-api`. Deocamdată serviciul e un hello-world; nu contează. Apoi **poarta**, care decide arhitectura în locul tău: schimbă un pas de scanare — pragul lui trivy, de exemplu — **într-un singur loc**, și rulează pipeline-ul ambelor servicii.

Gata când: Application-ul `curse-api` e Synced cu hello-world-ul înăuntru, **și** ambele servicii au primit schimbarea de scanare dintr-o singură modificare. Dacă a trebuit să modifci în două locuri, comanda a copiat pipeline-ul — și ai acum două fișiere care vor diverge. Pune-l să refacă: un workflow comun, chemat de fiecare serviciu.

Fișa 2, exercițiul 7, rezolvat structural: acolo aveai trei pipeline-uri scrise în momente diferite și întrebai „și celelalte două?”. Aici al doilea serviciu nu primește niciodată un pipeline al lui — primește o linie care îl cheamă pe singurul care există.

curse-api, și regula care trece acum prin două servicii

11

Ca la exercițiile 5–6, testele întâi, apoi:

```
Implementeaza curse-api dupa CONTRACT.md: pornirea unei curse cheama statii-api sa scoata o bicicleta din statia de plecare, inchiderea o pune in statia de sosire. Daca statii-api refuza, cursa nu se creeaza. Testele raman neschimbate.
```

Când totul e verde și deployat, **poarta** — un script, nu o părere:

```
Scrie invariant.sh: face 50 de operatii la intamplare pe API-ul din cluster – porniri, inchideri, inchideri repetate, statii inexistente, statii goale – apoi aduna bicicletele din statii cu cursele deschise si afiseaza suma. Trebuie sa fie 30.
```

Gata când: scriptul dă 30 de trei ori la rând. Apoi strici cu mâna: închiderea cursei nu mai cheamă `pune`. Scriptul trebuie să dea mai puțin de 30. Readu linia, `git diff gol`.

E cel mai greu exercițiu din fișă, și e normal să stea câteva zile. O bicicletă scoasă din stație de un serviciu și o cursă nesalvată de celălalt înseamnă o bicicletă dispărută: suma dă 29 și nimeni nu știe de ce. Agentul va spune „gata” înainte ca scriptul să spună 30. Scriptul are dreptate. Pune-l pe agent să caute până când cele două sunt de acord.

Skill-ul /invariant: verificarea pe care o alege singur

12

Fa un skill /invariant care ruleaza invariant.sh pe mediul local si raporteaza suma. Descrierea lui trebuie sa-l faca sa porneasca singur ori de cate ori se schimba o ruta care muta biciclete, fara sa-l chem eu.

Poarta pe descriere — într-o sesiune nouă, fără să pomenеști skill-ul:

Adauga in statii-api o ruta care muta 5 biciclete dintr-o statie in alta.

Gata când: după ce a scris ruta, a rulat verificarea **singur** și ți-a raportat suma. Și ai citit ce a găsit: transferul într-o stație aproape plină trece de 30 pe o stație și sub 0 pe cealaltă? Dacă skill-ul nu a pornit, nu codul e de vină — descrierea lui nu s-a potrivit cererii. Rescrie descrierea, nu cererea.

Clientul, pe contractul real

13

Scrive velo-web: pagina cu statiile si cursele deschise, pornire si inchidere de cursa. URL-urile celor doua API-uri se citesc la runtime dintr-un config/config.json servit langa pagina, NU din variabile de build. Rutele le iei din CONTRACT.md, nu le inventezi.

Apoi /serviciu-nou velo-web, și clientul intră în lanț ca celelalte două. **Poarta:** oprește statii-api — dar **prin repo:** replicas: 0 în values, commit, push, aștepți ArgoCD.

Gata când: cu backendul oprit, clientul arată o **eroare**, nu datele de dinainte și nu date inventate. Apoi replicas: 1, același drum, datele revin. Dacă prima ta pornire a fost să scrii kubectl scale, întoarce-te la exercițiul 1: gardul te-ar fi oprit, dar regula e pentru tine, nu doar pentru agent.

Partea IV — Harta, codul și clusterul: cine păzește ce

Subagentul harta-vs-cod

14

Fa un subagent "harta-vs-cod". Citeste CLAUDE.md, CONTRACT.md si cele patru repo-uri. Intoarce trei liste: (a) ce spune harta si codul nu face, (b) ce face codul si harta nu spune, (c) ce afirma harta si nu se poate verifica din niciun fisier. Nu propune fix-uri.

Gata când: lista (b) **nu e goală** — ruta de transfer de la exercițiul 12 nu e în CONTRACT.md, iar expirarea tokenului de la exercițiul 7 e un candidat bun pentru (c). Pentru fiecare rând decizi: harta se actualizează sau codul se scoate. „Amândouă sunt în regulă” nu e o decizie.

De ce al doilea rol: agentul care a scris ruta e ultimul care observă că n-a trecut-o în contract. Subagentul nu e mai deștept — doar nu a scris-o el.

Hook-ul care ține harta sinceră

15

Un singur criteriu, altul decât cele din fișele 1 și 2:

```
Pune un hook inainte de git commit: daca diff-ul adauga o ruta HTTP noua
intr-un serviciu, sau un Application nou in velo-gitops, si nici
CONTRACT.md nici CLAUDE.md nu sunt in acelasi commit, opreste commit-ul
cu exit 2 si spune ce lipseste.
```

Gata când: ai văzut amândouă rezultatele: rută nouă fără contract → oprit, cu motivul; aceeași rută plus o linie în `CONTRACT.md` → trece. Două teste, nu unul. De aici încolo lista (b) a subagentului ar trebui să rămână goală — verific-o peste o săptămână.

Distruge clusterul. Reconstruiește-l din repo.

16

În Rancher Desktop: *Reset Kubernetes*. Clusterul dispare cu tot ce era în el. Apoi **exact cele două comenzi** de la exercițiul 8 — `helm install` pentru ArgoCD și `apply` pe `root.yaml` — și nimic altceva. Verificarea finală nu e o sesiune, e o comandă într-un script:

```
claude -p "Verifica: toate Application-urile Synced si Healthy, cele trei
ingress-uri raspund, invariant.sh da 30. Raspunde OK sau lista problemelor."
```

Gata când: răspunsul e OK și **numărul de comenzi date de mâna ta a fost doi**. Dacă a fost nevoie de a treia — un namespace, un secret, un `hosts` uitat — aia e gaura din repo: o scrii în repo, distrugi din nou, numeri din nou.

Ce ai construit, de fapt: nu două servicii și un client — o platformă la care întrebarea „pot s-o refac doar din repo-uri?” are un răspuns, pentru că ai pus-o înainte de al doilea serviciu, nu după al douăzeci și șaptelea Application. Pe `ms\` întrebarea asta încă n-are răspuns. Acum știi ce costă să i-l dai: două comenzi, dacă restul e în repo.

Ce au în comun toate cele 16: nicio verificare din fișă nu a fost scrisă după lucrul pe care îl verifică. Gardul înainte de fișier, harta înainte de repo, testul înainte de cod, lanțul înainte de al doilea serviciu, scriptul înainte de „gata”. Agentul a scris aproape tot codul, și tot codul a trecut printr-o poartă pe care ai putut-o vedea cu ochii tăi: un run colorat, un pod, un `curl`, o sumă. Partea grea n-a fost codul. A fost să decizi, de fiecare dată, ce trebuie să existe înainte.